

Parte 3. Interpretación de comandos

Responde esta parte en el documento parte_3/parte_3.pdf. Incluye cada planteamiento y su explicación.

1. Analiza

Planteamiento: Explica qué ocurre en cada instrucción del siguiente bloque:

```
git status
git add README.md
git commit -m "Actualiza documentación"
git push
```

Cada instrucción cumple una etapa secuencial y necesaria dentro del flujo de trabajo estándar de Git y GitHub:

1. `git status`: Muestra el estado actual del repositorio de trabajo. Permite visualizar qué archivos han sido modificados, creados o si existen elementos pendientes por rastrear (*untracked*), indicando si el área de trabajo está limpia o si hay cambios listos para ser preparados.
2. `git add README.md`: Agrega de forma explícita el archivo README.md al área de preparación (Staging Area). Esto le indica a Git que este archivo específico está marcado y listo para ser incluido en la próxima instantánea o registro histórico.
3. `git commit -m "Actualiza documentación"`: Crea un registro permanente (commit) en el historial local del repositorio utilizando los archivos que se encontraban en el área de preparación. El parámetro `-m` añade un mensaje descriptivo ("Actualiza documentación") que documenta formalmente el propósito de las modificaciones realizadas.
4. `git push`: Envía los commits que se han acumulado y guardado en el repositorio local hacia el repositorio remoto alojado en la plataforma de GitHub (en su rama correspondiente), actualizando la nube con los últimos avances del desarrollo.

2. Identifica qué falta

Caso A

- Secuencia dada:
Modificar archivo

↓

```
git add .
```

↓

¿?

↓

git push

- Operación faltante: git commit -m "Mensaje descriptivo"
- El comando git add . transfiere los archivos modificados al área de preparación (Stage), pero para registrar formalmente esos cambios en el historial de Git es obligatorio ejecutar un commit. No es posible realizar un push efectivo ni con sentido cronológico si los cambios no han sido consolidados previamente mediante un commit local.

Caso B

- Secuencia dada:
Repositorio GitHub

↓

¿?

↓

Repositorio local

- Operación faltante: git clone <url> (o en su defecto, si ya existe una carpeta vinculada localmente, git pull).
- Para traer un proyecto desde un repositorio remoto en GitHub por primera vez hacia una computadora local, se requiere la operación de clonación (git clone). Si el repositorio local ya existiera previamente y solo se quisieran rescatar actualizaciones posteriores, se emplearía git pull.

Caso C

- Secuencia dada:
Repositorio remoto actualizado

↓

¿?

↓

Repositorio local actualizado

- Operación faltante: git pull (o bien la combinación técnica de git fetch seguida de git merge).
- Cuando el repositorio remoto en GitHub ha recibido nuevos commits por parte de otros colaboradores, la terminal de tu computadora local se encuentra desactualizada. El comando git pull se encarga de descargar e integrar automáticamente esas modificaciones remotas hacia tu entorno de trabajo local.